

1.What If There Were No Operating System? Imagine a computer without an operating system. Identify the problems a user would face and investigate how OS features solve them. Demonstrate at least three features using Linux and present a “Without OS vs With OS” comparison.

Answer:

An Operating System (OS) is the main software that helps the user communicate with the computer hardware. Examples of operating systems are Linux, Windows and macOS.

If there were no operating system, using a computer would be very difficult. The user would have to communicate directly with the hardware, which is not practical for normal users.

Problems Without an Operating System

- It would be difficult to run applications.
- The user would have to directly control hardware.
- Managing files would be difficult.
- There would be no easy way to manage memory and CPU.
- Devices such as keyboards, printers and USB drives would be difficult to use.
- Security and user permissions would be difficult to manage.

Without OS

vs

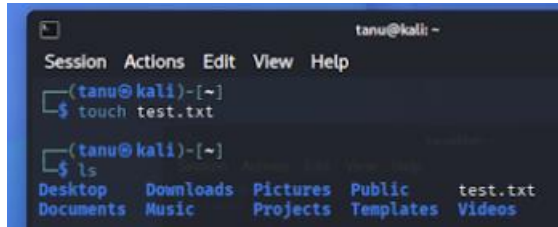
With OS

User has to deal directly with hardware	OS provides an easy interface
Difficult to manage files	Files can be created, copied, moved and deleted easily
Difficult to run programs	Programs can be executed easily
Hardware must be controlled manually	OS manages hardware devices
No proper memory management	OS manages RAM and other resources
No proper user permissions	OS provides users and permissions

Linux Features Demo

Feature 1: File Management

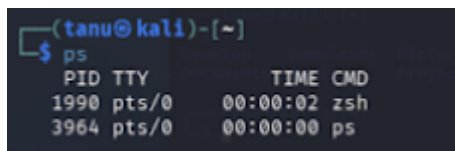
Linux provides commands for creating and managing files.

A terminal window titled 'tanu@kali: ~' with a menu bar (Session, Actions, Edit, View, Help). The user enters 'touch test.txt' and then 'ls'. The output of 'ls' shows a directory listing with icons for Desktop, Documents, Downloads, Music, Pictures, Projects, Public, Templates, Videos, and a file named test.txt.

```
tanu@kali: ~  
Session Actions Edit View Help  
(tanu@kali)-[~]  
$ touch test.txt  
(tanu@kali)-[~]  
$ ls  
Desktop Downloads Pictures Public test.txt  
Documents Music Projects Templates Videos
```

Feature 2: Process Management

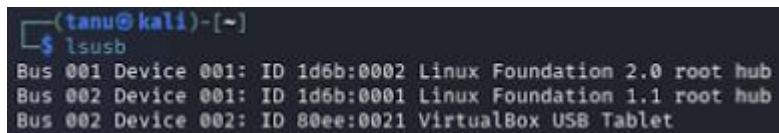
Linux allows us to see running processes.

A terminal window titled '(tanu@kali)-[~]' showing the output of the 'ps' command. It lists the PID, TTY, TIME, and CMD for the current shell (zsh) and the 'ps' command itself.

```
(tanu@kali)-[~]  
$ ps  
PID TTY TIME CMD  
1990 pts/0 00:00:02 zsh  
3964 pts/0 00:00:00 ps
```

Feature 3: Hardware and Device Management

Linux provides commands to get information about the system hardware.

A terminal window titled '(tanu@kali)-[~]' showing the output of the 'lsusb' command. It lists USB buses and devices with their IDs and descriptions.

```
(tanu@kali)-[~]  
$ lsusb  
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub  
Bus 002 Device 001: ID 1d6b:0001 Linux Foundation 1.1 root hub  
Bus 002 Device 002: ID 80ee:0021 VirtualBox USB Tablet
```

2. Imagine your team is setting up a cybersecurity laboratory. Explore at least five Linux distributions relevant to cybersecurity and recommend suitable distributions for different roles such as penetration testing, digital forensics, privacy, and security monitoring. Present your findings creatively as a comparison chart, poster, infographic, or “OS selection guide”.

Different Linux distributions are designed for different purposes. Some are mainly used for penetration testing, while others are better for digital forensics, privacy or security monitoring.

Some OS distributions

Linux Distribution	Main Purpose	Suitable Role	Important Features
kali Linux	Penetration testing	Ethical Hacker / Pen Tester	Large collection of security tools
Parrot Security OS	Security testing and privacy	Pen Testing / Security Research	Lightweight and security-focused
Tails	Privacy and anonymity	Privacy / Secure Browsing	Runs from USB and focuses on privacy
BlackArch Linux	Advanced security testing	Security Researchers / Pen Testers	Large collection of security tools
Security Onion	Network security monitoring	SOC / Security Analyst	Network monitoring and threat detection

1. Kali Linux

Kali Linux is one of the most commonly used Linux distributions for cybersecurity. It contains many tools for penetration testing, vulnerability analysis, network testing and password security.

Best suited for:

- Penetration Testing
- Ethical Hacking
- Security Testing

2. Parrot Security OS

Parrot Security OS is another Linux distribution designed for security and privacy. It can be used for penetration testing, digital forensics and security research.

It is also relatively lightweight compared to some other security-focused distributions.

Best suited for:

- Penetration Testing
- Security Research
- Privacy

3. Tails

Tails is designed mainly for privacy and anonymity. It can be run from a USB drive without installing it permanently on the computer.

It is useful when privacy is more important than having a large collection of security testing tools.

Best suited for:

- Privacy
- Anonymous browsing
- Secure temporary sessions

4. BlackArch Linux

BlackArch is based on Arch Linux and is designed mainly for penetration testing and security research. It provides a large collection of cybersecurity tools.

It is more suitable for users who already have some Linux knowledge.

Best suited for:

- Advanced Penetration Testing
- Security Research
- Vulnerability Testing

5. Security Onion

Security Onion is focused on defensive cybersecurity. It is useful for monitoring networks, detecting suspicious activity and investigating security incidents.

It is more suitable for a Security Operations Center (SOC) environment than for normal penetration testing.

Best suited for:

- Security Monitoring
- SOC
- Network Security
- Threat Detection

“There is no single Linux distribution that is best for every cybersecurity task. The choice depends on the role. For a beginner-friendly penetration testing environment, Kali Linux would be my first choice. For defensive security and network monitoring, Security Onion would be more suitable.”

3. Exit status:

Execute at least five Linux commands that produce both successful and unsuccessful results. Investigate their exit status using ` \$? `. Create a simple visual/table showing Command → Result Exit Status → Meaning. What can a script learn from an exit status?

In Linux, every command returns an exit status after it finishes executing.

The exit status tells us whether the command was successful or not.

- 0 → Command was successful
- Non-zero value → Command failed or produced some error

The special variable \$? stores the exit status of the most recently executed command.

1. Successful command

```
(tanu@kali)-[~]
$ pwd
/home/tanu

(tanu@kali)-[~]
$ echo $?
0
```

Meaning: pwd executed successfully.

2. Successful command

```
(tanu@kali)-[~]
$ ls
Desktop  Downloads  Pictures  Public  test.txt
Documents Music      Projects  Templates Videos

(tanu@kali)-[~]
$ echo $?
0
```

3. Unsuccessful command

```
(tanu@kali)-[~]
$ ls abc123
ls: cannot access 'abc123': No such file or directory

(tanu@kali)-[~]
$ echo $?
2
```

4. Successful command

```
(tanu@kali)-[~]  
$ mkdir testfolder  
  
(tanu@kali)-[~]  
$ echo $?  
0
```

5. Unsuccessful command

```
(tanu@kali)-[~]  
$ cat abc.txt  
cat: abc.txt: No such file or directory  
  
(tanu@kali)-[~]  
$ echo $?  
1
```

Command → Exit Status:

command	Result or exit status	meaning
pwd	0	Successful
Ls	0	Successful
Ls abc.txt	2	File or dir not found
Mkdir testfolder	0	Directory created successfully
Cat abc123.txt	1	File not found

What Can a Script Learn From an Exit Status?

script can use the exit status to decide what to do next.

exit statuses help scripts make decisions, detect errors and continue or stop depending on the result of a command.